



Resumen Clase 03/10 - Administración de E/S

1. Conceptos generales de E/S

Tipos de dispositivos

Dispositivos de almacenamiento (discos rígidos, SSD, cintas)

Dispositivos de comunicación

Dispositivos de interfaz de usuario

Componentes

Dispositivo físico

Controlador (Controller):

Driver:

Categorías según funcionamiento

Dispositivos de caracteres (char devices)

Dispositivos de bloques (block devices)

2. Mecanismos de comunicación entre CPU y dispositivos

E/S síncrona vs asíncrona

Síncrona:

Asíncrona:

Métodos de acceso

Polling:

Interrupciones:

DMA (Direct Memory Access):

Comparativa

3. Subsistema de E/S del Sistema Operativo

¿Qué es?

Estructura

¿Qué funciones cumple el subsistema de E/S?

Proveer una API uniforme

4. Drivers

API típica de un driver

5. Funciones del kernel para drivers

6. Concurrencia en los drivers

7. Scheduling de E/S a disco

8. Gestión del disco

1. Conceptos generales de E/S

Tipos de dispositivos

Dispositivos de almacenamiento (discos rígidos, SSD, cintas)

- Permiten guardar información de manera persistente.
- Ejemplos: discos rígidos (HDD), discos de estado sólido (SSD), unidades de cinta, disquetes, CD/DVD, memorias USB.
- Pueden ser **locales** o **remotos** (por red):
 - **NAS (Network Attached Storage)**: dispositivos accesibles por red con protocolos como NFS, CIFS o AFS.
 - **SAN (Storage Area Network)**: red especial de almacenamiento de alto rendimiento, con protocolos específicos.

Dispositivos de comunicación

- Intercambian información entre computadoras o sistemas.
- Ejemplos: placas de red, módems, interfaces Bluetooth, puertos serie, Wi-Fi.

Dispositivos de interfaz de usuario

- Permiten la interacción directa entre usuario y computadora.
- Ejemplos: teclado, mouse, pantalla, micrófono, parlantes, cámaras.

Componentes

Dispositivo físico

- El hardware que realiza la función (el disco, teclado, impresora, etc.).

Controlador (Controller):

- Componente mecánico y/o electrónico que trabaja como una interfaz entre un dispositivo y el driver.
- Traduce las órdenes del sistema a señales eléctricas que el dispositivo entiende.
- Puede tener registros de control, estado y datos.

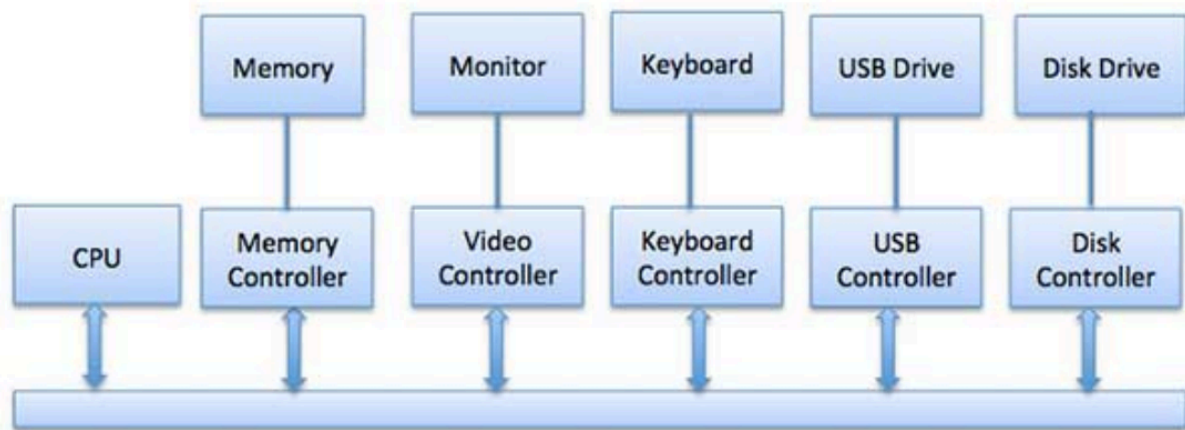


Figure: Controladores de hardware (*Controllers*)

Driver:

- Módulos de software que pueden ser añadidos al SO para manejar los dispositivos de E/S.
- Corren con máximo privilegio: pueden hacer colgarse a todo el sistema.
- De ellos depende el rendimiento de E/S, que es fundamental para el rendimiento combinado del sistema.

Categorías según funcionamiento

Dispositivos de caracteres (**char devices**)

- Dispositivos en los cuales se transmite la información byte a byte.
- Ejemplos: mouse, teclado, terminales o puerto serie. Debido a su acceso secuencial (byte a byte) no soportan acceso aleatorio y no utilizan cache.

Dispositivos de bloques (**block devices**)

- Dispositivos en los cuales se transmite la información en bloque.

- Ejemplos: disco rígido, flash memory o CD-ROM. Permite el acceso aleatorio y por lo general utilizan un buffer (cache).
-

2. Mecanismos de comunicación entre CPU y dispositivos

E/S síncrona vs asíncrona

Síncrona:

- La ejecución de la CPU que solicita la E/S, espera por su culminación.

Asíncrona:

- Cada E/S procede concurrentemente con la ejecución del CPU que la solicita.

Métodos de acceso

Polling:

- La CPU revisa periódicamente si el dispositivo terminó.
- Utiliza un registro de status.

Interrupciones:

- El controller coloca una señal en el bus, cuando necesita atención de la CPU.
- Utiliza los handlers o rutinas de software para manejar cada tipo de interrupción.

DMA (Direct Memory Access):

- Transfiere grandes volúmenes directamente entre dispositivo y memoria sin intervención de CPU.

Comparativa

- Polling: simple pero ineficiente (consume CPU).

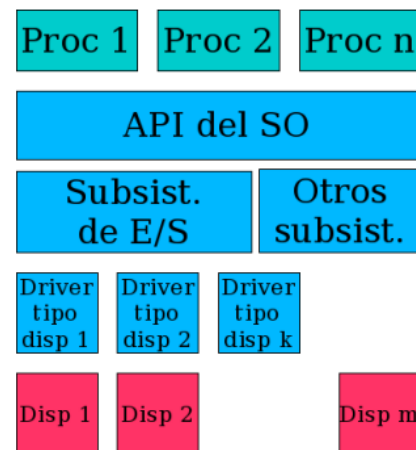
- Interrupciones: eficiente, pero con cambios de contexto no deterministas.
- DMA: eficiente para grandes transferencias, requiere hardware dedicado.

3. Subsistema de E/S del Sistema Operativo

¿Qué es?

El **subsistema de Entrada/Salida (E/S)** es el **conjunto de componentes del sistema operativo que se encargan de gestionar los dispositivos**: discos, teclado, pantalla, red, impresoras, etc.

Es decir, es la **parte del kernel que se interpone entre las aplicaciones y el hardware**.



- ◆ Idea central: el subsistema de E/S oculta la complejidad del hardware y proporciona una interfaz uniforme para todos los dispositivos, aunque internamente sean muy distintos.

Estructura

- Scheduling.
- Buffering.
- Caching.
- Spooling.
- Manejo de errores.

¿Qué funciones cumple el subsistema de E/S?

Proveer una API uniforme

- Para el usuario o programador, **todos los dispositivos parecen archivos**.
- Por eso se usan las mismas llamadas del sistema:
 - `open()` – abrir un archivo o dispositivo
 - `read()` / `write()` – leer o escribir datos
 - `close()` – cerrar el descriptor
 - `seek()` – mover el puntero de lectura/escritura

💡 En Unix/Linux, todo es un archivo, incluso los dispositivos (`/dev/sda`, `/dev/tty`, etc.).

4. Drivers

- **Definición:** Módulos de software dentro del kernel que manejan dispositivos específicos. Conocen la estructura de registros, bits de control y protocolo del hardware.
- **Importancia:** afectan estabilidad y rendimiento del sistema.
- **Roles**
 - Aceptan pedidos del software (API del SO)
 - Interactúan con el controlador del hardware
 - Manejan errores y concurrencia

API típica de un driver

- Un driver debe implementar los siguientes procedimientos para ser cargado por el sistema operativo.
 - `int driver_init()` Invocada durante la carga del SO.
 - `int driver_open()` Invocada al solicitarse un open.
 - `int driver_close()` Invocada al solicitarse un close.
 - `int driver_read(int *data)` Invocada al solicitarse un read.
 - `int driver_write(int *data)` Invocada al solicitarse un write.

- `int driver_remove()` Invocada durante la descarga del SO
-

5. Funciones del kernel para drivers

- `OUT(IO_address, data)` → escribir en registro de E/S
 - `IN(IO_address)` → leer de registro de E/S
 - `request_irq(i rq, handler)` → registrar rutina de interrupción
 - `free_irq(irq)` → liberar interrupción
 - `copy_from_user()` / `copy_to_user()` → acceso seguro entre espacio de usuario y kernel
-

6. Concurrencia en los drivers

- El driver corre en contexto de proceso
 - Múltiples procesos pueden usar el mismo driver → **condiciones de carrera**
 - Solución:
 - Mutex o semáforos para secciones críticas
 - Inicialización en `driver_init()`
 - No usar bibliotecas de usuario — sólo funciones del kernel
-

7. Scheduling de E/S a disco

- **Objetivo:** optimizar el acceso al disco minimizando movimientos de cabezal.
- **Tiempos importantes**
 - *Seek time* (búsqueda del cilindro)
 - *Rotational latency* (espera de rotación)
 - *Transfer rate* (ancho de banda)
- **Políticas de planificación**
 - **FCFS (First Come First Served):** simple pero poco eficiente

- **SSTF (Shortest Seek Time First):** atiende el pedido más cercano
 - **SCAN (Elevator):** recorre en un sentido y luego el otro
 - Variantes prácticas combinan prioridades y políticas mixtas.
-

8. Gestión del disco

- **Formateo:** marca sectores con prefijos/postfijos para detección de errores.
 - **Booteo:** programa en ROM carga el sector inicial del disco (bootloader).
 - **Bloques dañados:** remapeo o manejo por software (ej: FAT).
-

9. Spooling

- **Definición:** es una forma de manejar a los dispositivos que requieren acceso dedicado en sistemas multiprogramados.
 - **Mecanismo:** los trabajos se almacenan en cola, y designar un proceso que los desencole a medida que el dispositivo se libere.
 - El kernel no lo gestiona directamente; es responsabilidad del usuario o proceso spooler.
 - El nombre viene de Simultaneous Peripheral Operation On-Line.
-

10. Copias de seguridad y redundancia

- **Backups**
 - Tipos: **total, incremental, diferencial**
 - Propósito: proteger la información ante pérdida o corrupción.
- **RAID (Redundant Array of Inexpensive Disks)**
 - **RAID 0:** *stripping* – sin redundancia, alto rendimiento
 - **RAID 1:** *mirroring* – duplicación total
 - **RAID 0+1:** combinación de los dos anteriores
 - **RAID 3/4/5/6:** usan bits de paridad distribuidos para tolerancia a fallos

- **RAID 5:** más usado (distribución de datos y paridad, tolera falla de un disco)
- **RAID 6:** dos bloques de paridad, tolera falla de dos discos
- **RAID ≠ Backup:** RAID protege de fallos físicos, no de errores lógicos o borrados.